



Mentalidade de Programação

Como escrever código que funciona no mundo real

Quando procuramos soluções rápidas para problemas de código, encontramos uma abundância de recursos online sobre práticas eficazes de programação, comentários de código ou documentação. No entanto, para muitos de nós, é difícil contextualizar essas informações em soluções úteis para o nosso próprio trabalho. Isso é uma desvantagem de como a programação é ensinada nas escolas, com um foco tremendo em transmitir o conhecimento técnico necessário para resolver problemas de resposta fechada.

Conteúdo traduzido e adaptado pelo Labdiv de um artigo criado pelo [MIT Electrical Engineering and Computer Science Communication Lab](#).

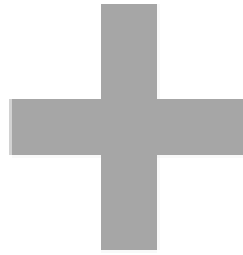
Coding Mindset

Neste artigo, queremos ensinar uma mentalidade de programação que complementa a expertise técnica, ajudando a contextualizar as melhores práticas genéricas para problemas complexos do mundo real.





Technical Expertise



Coding Mindset

A programmer's three high-level goals are to write code that...

- ⚙ Solves a specific problem
- ⚙ Is easy to read
- ⚙ Is maintainable and extendable

Contrariando o senso comum, a mentalidade de programação não é como o gosto de um bom vinho; não é algo adquirido apenas com o tempo. Ela pode ser ensinada sistematicamente também a iniciantes. Ao mesmo tempo, programadores de *qualquer* nível de experiência se beneficiam ao praticar essa mentalidade. A mentalidade de programação nos ajuda a aplicar a expertise técnica desenvolvida na escola, mantendo em vista nossos objetivos como programadores, resultando em decisões que consideram as complexidades do mundo real.

Ao levar em conta as demandas de projetos com múltiplos códigos interagindo e colaboradores que precisam usar e adaptar nosso código, podemos nos esforçar para tomar decisões de programação que priorizem clareza, modularidade e reutilização.

Os três objetivos principais de um programador são escrever código que...

1. Resolva um problema específico
2. Seja fácil de ler e entender
3. Seja sustentável e extensível

O processo de tomada de decisão na programação é orientado tanto pela expertise técnica quanto pela mentalidade de programação. A expertise técnica, por si só, não é suficiente quando precisamos escolher a solução certa entre várias abordagens possíveis para resolver um problema. A mentalidade de programação, por outro lado, nos ajuda a escolher a solução que melhor atende aos objetivos principais, que às vezes podem ser contraditórios.

Por exemplo, você pode se sentir tentado a usar o Algoritmo X Inovador para resolver o Problema Y, mas então percebe que X apresentaria problemas de escalabilidade quando você considera a Situação Z do Mundo Real. Então, você decide dar um passo atrás e reavaliar seus objetivos principais, percebendo que, nesse caso, é mais importante priorizar a extensibilidade. Isso faz com que você opte pelo Algoritmo W Menos Inovador, que está mais alinhado com seus objetivos. Seus colaboradores ficam satisfeitos. Você ganha o prêmio de Programador Realista do Ano!

Uma mentalidade de programação eficaz começa ao se perguntar se os três objetivos principais como programador estão sendo atendidos.

Se você não consegue satisfazer os três objetivos de uma vez, a mentalidade o lembra de perguntar quais um ou dois são mais importantes para você e seus colaboradores. Com prática, se tornará algo natural avaliar suas escolhas com essas perguntas.

A mentalidade é de aplicação prática e pode ser reforçada com exemplos. Abaixo, expandimos cada um dos objetivos principais e apresentamos ilustrações de como aplicar uma mentalidade de programação eficaz.



1. Escreva código que resolva um problema específico

Evite tentar escrever um único código que possa resolver uma ampla gama de problemas. Isso pode parecer contraintuitivo, então vejamos um exemplo: você já viu um faz-tudo usar um canivete suíço para consertar sua mobília, em vez de usar uma ferramenta específica escolhida de um conjunto de ferramentas especializado? Podemos aplicar o mesmo raciocínio à programação: comece escrevendo código que resolva um problema específico. Quando necessário, o código pode ser estendido para resolver problemas semelhantes ao adicionar a funcionalidade necessária. Isso é semelhante a ampliar a funcionalidade de um conjunto de ferramentas adicionando mais ferramentas.



Escrever código que resolva um problema específico facilita a extensão da funcionalidade de forma modular.

- Antes de começar a programar, articule uma descrição clara do problema que você deseja resolver.
- Sua documentação e/ou README devem começar com essa descrição do problema.
- Identifique e modularize os diferentes componentes da sua solução que podem ser potencialmente reutilizados para resolver problemas semelhantes.

2. Escreva código que seja fácil de ler e entender

Programadores muitas vezes superestimam sua capacidade de compreender código escrito com poucos comentários e estilos inconsistentes. Estudos mostram que, após um período de duas semanas, um programador médio não consegue dizer se um trecho de código sem comentários e anonimamente apresentado foi escrito por ele ou por outra pessoa – o que reduz a capacidade de entender e reutilizar o código. Isso destaca a necessidade de adotar padrões que tornem o código mais fácil de ler e entender.

2.1 Convenções de codificação

2.1.1. Variáveis e funções são para o código o que substantivos e verbos são para a comunicação.

- Cada peça de informação que precisa ser armazenada deve ser uma variável.
O nome de uma variável deve descrever seu conteúdo.
- Cada conjunto de ações realizadas no código deve estar contido em uma função.
O nome de uma função deve descrever as ações que ela executa.

Assim como qualquer forma de comunicação se torna ambígua quando usamos um único substantivo ou verbo para se referir a vários objetos ou ações, ler código torna-se difícil quando variáveis e funções recebem nomes que não são únicos e específicos.

2.1.2. 'Hard coding' é um pecado capital da programação e deve ser evitado a todo custo.

'Hard coding' significa inserir dados ou parâmetros diretamente no código, em vez de lê-los ou gerá-los a partir de uma fonte externa. Isso faz com que o código seja excessivamente personalizado para um problema ou configuração específicos, dificultando sua extensão para um conjunto maior de problemas. Para evitar 'hard coding', identifique quais parâmetros provavelmente variam de acordo com o problema ou conjunto de dados que você está tratando. Configure o código para ler essas informações em tempo de execução a partir de um script de configuração, entrada via linha de comando, ou algo similar.



All rights reserved by Reddit user Murmani or original creator.
Reproduced here for educational purposes only.

Constantes que não são específicas de um problema devem ser definidas como variáveis com nomes descritivos. Por exemplo:

- Seu eu futuro pode ficar confuso se encontrar `if (resposta = 3.841)`. O valor 3.841 é um parâmetro inventado?
- Considere, em vez disso: `chi_square_05 = 3.841`. Agora fica claro que você está testando significância estatística, e o número tem relevância matemática.

2.1.3. Use um estilo de codificação consistente que aproveite nossa habilidade visual para reconhecer padrões e ordem espacial.

[Clique aqui para ler o artigo do CommKit sobre como desenvolver e usar um estilo de codificação eficaz.](#)



2.2 Comentários e documentação

[KitDiv \(dicas práticas\)](#)[Agendar uma mentoria](#)

Comentários e documentação têm propósitos distintos e devem, juntos, fornecer ao usuário final e ao desenvolvedor todas as informações necessárias para utilizar, manter e expandir o código facilmente. Comentários explicam escolhas de programação que não são óbvias, enquanto a documentação descreve o que o código faz.

2.2.1. Comentários no código-fonte explicam escolhas de programação que não são evidentes.

O objetivo é deixar pistas úteis para uma versão futura de você mesmo ou para outro desenvolvedor que queira manter e expandir o código.

Evite encher o código com comentários óbvios, como “armazena o valor 10”. Em vez disso, explique o raciocínio não evidente. Por exemplo:

- Você escolheu o algoritmo de ordenação bubble sort para ordenar uma lista pré-ordenada à qual um elemento é adicionado a cada iteração. No entanto, outro desenvolvedor pode se perguntar por que um algoritmo assintoticamente mais eficiente, como quicksort, não foi usado.
- Solução: Deixe um comentário explicando que o bubble sort foi preferível neste caso específico, pois é o algoritmo mais rápido para listas quase ordenadas.

2.2.2. Uma documentação de código eficaz deve apresentar claramente...

1. O problema que o código pretende resolver
2. A metodologia usada para resolver o problema
3. Onde encontrar referências relevantes, como o algoritmo, estudos de validação ou estudos de benchmarking
4. Onde procurar no código as partes que leem a entrada, o algoritmo que resolve o problema, a escrita da saída, etc.

3. Escreva código que seja sustentável e expansível



3.1 Reduza duplicação de código e aumente a reutilização. Agendar uma mentoria

Esse é o mantra da programação orientada a objetos, que impulsionou o crescimento da indústria de ciência da computação nos anos 90, após uma década de progresso modesto com linguagens de programação estruturadas que tinham muita duplicação de código. A duplicação de código aumenta significativamente o risco de corrupção quando mudanças não são propagadas para todas as instâncias duplicadas.

Para criar código reutilizável, desenvolva funções que executem tarefas únicas (lembre-se: funções são como verbos de ação). Além disso, é uma boa prática [refatorar o código](#) em intervalos regulares para manter a base de código enxuta e melhorar ainda mais a reutilização. Como regra geral, se você não consegue encontrar um nome descritivo simples (verbo de ação) para uma função, isso indica a necessidade de refatoração.

3.2 Modularize a organização do código e dos dados.

Modularizar o código e a funcionalidade para evitar duplicação fornece uma estrutura clara, facilitando a navegação e reutilização do código. As chances de corrupção de dados podem ser bastante reduzidas mantendo os dados localizados espacial e temporalmente. Em outras palavras, busque a menor distância possível entre a primeira aparição de uma variável no código e o local onde ela é realmente utilizada. A melhoria na localidade facilita encontrar e corrigir bugs, tanto para você no futuro quanto para outros desenvolvedores.

- Limite conscientemente o escopo ativo de cada variável, utilizando seu conteúdo o mais próximo possível do local de sua geração.
- Coloque todas as entradas e parâmetros de código em um arquivo de configuração, carregado durante a inicialização. Isso permite a execução dinâmica do código e evita hard coding.
- Para ajudar a modularizar seus pensamentos sobre testes e depuração, você pode:
 - Utilizar [assertivas de código](#) para garantir comportamentos físicos a todo momento. Assertivas interrompem a execução quando uma entrada ou estado inesperado é detectado.
 - Por exemplo, você pode usar assertivas para garantir que atributos como massa ou contagem de objetos não sejam negativos.
 - Aproveitar [testes automatizados](#) para garantir a qualidade contínua do código, especialmente útil em projetos grandes.
 - Traduzir cada bug semântico detectado em um [teste unitário](#), fazendo com que a base de testes automatizados cresça organicamente.

Conclusão

As estratégias práticas apresentadas neste artigo ajudarão você a exercitar e reforçar uma mentalidade de codificação que suporte seus objetivos de alto nível como programador. Em resumo:

- Sempre se lembre do problema específico que você está resolvendo. 💡
- Faça escolhas de nomenclatura e estrutura que reflitam esse problema específico. 🧩
- Evite duplicação de código e melhore a localidade espacial e temporal dos dados. ↺

Para saber mais sobre como colocar sua mentalidade de programação em prática, explore outros artigos sobre boas práticas para programadores:

- [Estilo de Código e Comentários](#)
- [Estrutura de Arquivos](#)

Ainda precisa de ajuda? [Marque um atendimento](#) com um de nossos mentores.

*LabDiv, Ed. Novo Milênio, Instituto de Física , Universidade de São Paulo , Brasil
Contato: labdiv@usp.br*